

TEAM 4

INTELLIGENT BIKE DASH CAM

SOFTWARE QUALITY PLAN

10/8/2026

OVERVIEW

1. Project Description

The Intelligent Bike Dash Cam (IBDC) project provides a mobile application for reviewing ride sessions and incident evidence received from a connected intelligent bike dash cam. Team 4, including Matthew Edlin, Colin Tugend, Jair Franco, and Christopher Paquin are developing the application workflow that connects to the device through Bluetooth Low Energy (BLE), retrieves event information and images, stores related records locally, and submits anonymous data to a server-based analytics platform for the Rob Dollar foundation who commissioned this application.

The objective is to give riders a dependable way to preserve session and incident evidence for the Rob Dollar foundation to help make cycling safer in Arizona by informing the public and policy makers on trends. This plan covers our TypeScript and React Native application, its Expo dependencies, its local database, and its interface with the IBDC device.

The IBDC device and its firmware are external dependencies. This plan verifies the behavior visible at our interface. It does not define camera detection accuracy or hardware reliability.

2. High-Level Requirements

The following numbered requirements define the application's quality baseline. Requirements will be referred to as R# where # is where they appear on the list.

- 1) The application shall display the connection state reported by the BLE adapter.
- 2) The application shall display an error for an unsupported device protocol version.
- 3) The application shall translate device status values received through the protobuf protocol.
- 4) The application shall recover pending events after reconnection.
- 5) The application shall associate every image with its originating event.

- 6) The application shall accept an image as complete only after validating its metadata against the received payload.
- 7) The application shall resume an interrupted image transfer from the first missing chunk.
- 8) The application shall acknowledge an event only after its required evidence has been saved following successful validation.
- 9) The application shall preserve valid incidents within ride session records.
- 10) The application shall display ride history according to the selected filters.
- 11) The application shall display ride history using the selected page size.
- 12) The application shall classify incident completeness using the required evidence fields defined below in the testing plan.
- 13) The application shall display saved incident changes when the user returns to the ride session screen.
- 14) The application shall export a readable PDF of one selected incident with the evidence fields defined below in the testing plan.
- 15) The application shall exchange device messages using the agreed protobuf schema.
- 16) The application shall transmit data to a server as defined by the forthcoming agreement with an outside team.

3. Quality Metrics

This section describes the quality metrics used to evaluate the IBDC application. Each metric identifies what will be measured, the target for acceptance, and the related requirements and tests. These metrics cover correct operation, reliability, performance, and ease of use.

Metric	Description	Acceptance Target	Related Req(s)
M1	Measures whether the application displays the correct connection state, identifies unsupported protocol versions, and retrieves pending events after reconnecting.	100% of connection, compatibility, and pending-event recovery tests pass.	R1, R2, R4
M2	Measures whether device messages follow the agreed protobuf schema and device status values are translated correctly.	100% of protocol tests pass. Translated status values match the expected values in the test messages.	R3, R15
M3	Measures whether images are assigned to the correct event, validated before completion, and	100% of image transfer tests pass. No incomplete or invalid image is marked complete. Resumed images	R5, R6, R7

	resumed from the first missing chunk after interruption.	match the original test images.	
M4	Measures whether event acknowledgment occurs only after the required evidence has been validated and saved.	100% of acknowledgment tests pass. No acknowledgment is sent when validation or saving fails.	R8
M5	Measures whether saved incidents remain associated with the correct ride session.	100% of incident relationship and persistence tests pass. No valid incident is lost or assigned to the wrong session.	R9
M6	Measures whether ride history follows the selected filters and page size.	100% of filtering and page tests pass, including combined filters, empty results, and page boundaries.	R10, R11
M7	Measures whether the application correctly identifies complete and incomplete incidents using the required evidence fields.	100% of incident completeness tests pass. Missing required evidence results in an incomplete status but missing optional information does not.	R12
M8	Measures whether the ride session screen displays saved incident changes when the user returns to it.	100% of saved-change refresh tests pass. Displayed information matches the saved incident record.	R13
M9	Measures whether an exported PDF contains the selected incident's evidence in a readable format.	100% of PDF report tests pass. Required evidence matches the selected incident, with no missing content, clipping, or overlapping text.	R14
M10	Measures whether the application transmits data according to the agreement with the outside team.	Acceptance criteria will be defined when the agreement establishes the required data, transmission behavior, and expected responses.	R16

The testing plan (Reference All Test Cases.xlsx) will define the cases used to evaluate each metric listed above.

Human User Experience Testing

Human user experience (UX) testing will evaluate how the application feels to use through participant feedback and observations. The findings will be summarized in a report to our sponsor, including difficulties users encountered and suggested improvements. These results will help guide design decisions but will not produce an automatic pass or fail for the application's user experience.

4. Testing Plan

Testing will verify that the application meets the requirements in Section 2 and the quality metrics in Section 3. We will use automated tests for application logic, installed-app tests for native functionality, and physical-device tests for BLE communication. Human UX testing will provide feedback about how the application feels to use. Each test will record its expected result, actual result, and any problems found.

Testing Environments

Automated tests will use Jest with the jest-expo preset. React Native Testing Library will support tests of screen content and user interactions. These tests will use controlled data and mocks for dependencies such as BLE hardware, GPS, database access, and local file operations.

Native integration tests will run in installed development builds on Android and iOS. These tests will exercise the actual database, filesystem, permissions, and other platform-dependent behavior. Android and IOS emulators will support navigation and layout testing, but physical phones connected to the IBDC device will be used to verify BLE communication. The current plan is for Jair and our sponsor to meet to set this up.

Passing a test with a mock implementation does not establish that the corresponding native feature works. Applicable tests will therefore be repeated on both Android and iOS, with the device model, operating system, application build, and relevant firmware version recorded. The passing operating system versions will be shared with our sponsor to present a view of what user devices he can expect the software to work on.

Test Setup

The existing test-case spreadsheet (Reference All Test Cases.xlsx) will provide the starting inventory of inputs, outputs, and expected results. Additional cases will be added for PDF reports, server communication, and any so-far-uncovered requirements. Each case will identify its related requirement and quality metric.

Tests will create their own data and operate independently. Reusable factory functions will provide consistent sessions, incidents, images, and device messages. Tests involving timestamps or random data will use controlled values so results can be repeated. Mocks, listeners, and temporary records will be reset between tests unless a unique situation is found to require a no-reset test which will be documented.

BLE Connection and Recovery Testing

Applies to: M1, R1, R2, R4

Test Descriptions:

- Test successful connections, failed connections, unexpected disconnects, and reconnection.
- Verify that the displayed connection state matches the BLE adapter.
- Supply an unsupported protocol version and confirm that the application displays an error.
- Create pending events while disconnected, then reconnect and verify that the application retrieves them without creating duplicate incidents or listeners.

We will run these cases with a controlled BLE mock, then repeat the applicable workflows with the physical IBDC device when able.

Rationale: These tests verify that users receive accurate connection information and can recover evidence captured while the application was disconnected.

Protobuf Communication Testing

Applies to: R3, R15

Test Descriptions:

- Use test messages based on the agreed protobuf schema.
- Decode known messages and compare the translated device status values with the expected values.
- Encode outgoing messages and verify that their contents match the protocol.
- Include malformed messages to confirm that invalid messages are handled without being treated as valid data.

Rationale: Testing the messages verifies the actual communication contract.

Image Transfer Integrity Testing

Applies to: R5, R6, R7

Test Descriptions:

- Transfer single and multiple images using known image data and verify that every image is associated with its originating event. Compare the new image with the original test image.
- Test missing chunks, duplicate chunks, out-of-order chunks from different images, incorrect event identifiers, invalid sequence numbers, and incorrect byte counts. Valid out-of-order chunks will be reordered by sequence number before completion is evaluated. Duplicate chunks must not duplicate image content.
- Interrupt a transfer, reconnect, and verify that the application resumes from the first missing chunk. Confirm that incomplete or invalid images remain incomplete.

Rationale: Image transfer failures can damage incident evidence. These tests exercise both successful transfers and the conditions most likely to produce incomplete or incorrectly assembled images.

Event Acknowledgment Testing

Applies to: R8

Test Descriptions:

- Record the order of image validation, evidence saving, and event acknowledgment. Verify that acknowledgment occurs only after all required evidence has passed validation and has been saved.
- Introduce validation failures, database errors, and file-saving failures. Confirm that the application withholds acknowledgment when any required step fails. After successful completion, verify the acknowledgment sent to the device and check the device's pending-event response according to the protocol.

Rationale: Premature acknowledgment could cause the device to treat an event as transferred before the application has preserved its evidence.

Incident Data Integrity Testing

Applies to: R9

Test Descriptions:

- Create multiple ride sessions containing different incidents and images. Verify that each session displays only its own incidents and that each incident references the correct images.
- Save records, restart the application, and confirm that valid incidents remain available within the correct sessions.
- Repeat database initialization and verify that existing valid records are preserved.

Rationale: Correct relationships are necessary to keep images attached to the ride during which the incident occurred.

Incident Data Integrity Testing

Applies to: R12

Test Descriptions:

- Create a complete incident containing at least one image. Verify that the application identifies it as complete.
- Remove each required field separately and verify that the incident becomes incomplete.
- Test empty and whitespace-only license plates. Leave optional comments, driver, vehicle, and injury fields empty and confirm that the incident remains complete.

Rationale: These tests verify that the completeness status accurately identifies incidents that still require user attention.

Ride History Display Testing

Applies to: R10, R11.

Test Descriptions:

- Create sessions with no incidents, complete incidents, and incomplete incidents. Test each filter separately and both filters together.

- When Has Incidents and Action Required are both enabled, only sessions satisfying both filters should appear.
- Test the available page sizes, including 10, 25, 50, 100, and All.
- Verify navigation between pages, first-page and last-page controls, empty results, and page resets after filter changes. Confirm that changing the page size produces the expected number of displayed sessions without skipping or duplicating records.

Rationale: These tests verify that users can locate the intended ride sessions and navigate the results consistently.

Incident Completeness Testing

Applies to: R12.

Test Descriptions:

- Create a complete incident containing at least one image, a selected best image, latitude, longitude, creation time, and a nonblank license plate. Verify that the application identifies it as complete.
- Remove each required field separately and verify that the incident becomes incomplete.

Rationale: These tests verify that the completeness status accurately identifies incidents that still require user attention.

Saved Incident Changes Testing

Applies to: R13.

Test Descriptions:

- Open an incident from a ride session, change its selected image or editable details, save, and return to the ride session screen. Verify that the displayed information matches the saved record and that any affected completeness indicator is updated.
- Repeat the process on Android and iOS. Check that the application displays changes without requiring a restart or manual refresh.

Rationale: Users need to see that their changes were saved and understand whether an incident still requires review.

Incident PDF Report Testing

Applies to: R14.

Test Descriptions:

- Export a PDF for one selected incident and open the resulting file. Compare its contents with the incident record. These fields will be confirmed with the sponsor before acceptance testing.
- Test empty optional fields, long comments, multi-page reports, different image orientations, and non-ASCII text. Verify that the report contains the correct incident's evidence and remains readable without missing text, overlapping content, or distorted images.
- Test generation errors, unavailable image files, export failures, and cancellation. Confirm that these conditions leave the source incident unchanged and that failed exports are not reported as successful. Run the applicable tests on both Android and iOS.

Rationale: A report must preserve the incident's information in a form that another person can read and use. Successful file creation alone does not establish that the report is correct.

Server Data Transmission Testing

Applies to: R16.

Test Descriptions:

- Pending

Develop these tests after the agreement with the outside team defines the server interface. The agreement must establish the required data, message format, destination, expected responses, and failure-handling behavior before expected results can be finalized.

Rationale: Server communication must be tested against an agreed interface.

R16 will remain pending verification until the agreement and corresponding acceptance tests are available.

Human UX Testing

Human UX testing will evaluate how understandable and comfortable the application is to use. We propose 2 rounds of participants from outside the development team. The first round will identify any problems, and the second will evaluate changes made in response to that feedback.

Participants will receive tasks without being shown the steps. Tasks will include finding an incident that needs review, selecting an image, saving incident details, locating an older ride, and exporting an incident PDF.

Observers will record whether participants finish independently, where they need help, how long tasks take, and what causes confusion. Testers will rate task difficulty and describe their confidence that evidence was saved.

The team will provide the sponsor with a report summarizing observations, participant feedback, and recommended improvements. These findings will guide design decisions and will not create an automatic UX pass or fail determinations.

Test Results and Review

Each test result will identify the case, related requirement, application build, environment, expected result, actual result, and supporting evidence.

Errors will include reproduction steps and their effect on the user or application. After a correction, the affected tests will be repeated. Automated tests will run throughout development and ensure code changes don't break existing functionality.

The final review will compare functional results with the acceptance targets in Section 3. Human UX findings will be presented separately to the sponsor as feedback for review and improvement.